

PyTorch Study Notes — Basic to Advanced

A learning-oriented guide: concept → code → practice, for each topic.

How to use this document

Each section has three parts:

- **Concept** — what it is and why it matters
- **Code** — a minimal, runnable example
- **Try it yourself** — small exercises to reinforce the idea

Work through them in order. Don't skip the exercises — typing the code yourself (not copy-pasting) is what makes it stick.

Module 1: Tensors — The Building Block

Concept: A tensor is just a multi-dimensional array (like a NumPy array) that can also live on a GPU and track gradients. Everything in PyTorch — data, weights, activations — is a tensor.

```
python

import torch

a = torch.tensor([1, 2, 3])
b = torch.zeros(2, 3)
c = torch.rand(2, 3)           # uniform random
d = torch.randn(2, 3)          # normal random
e = torch.arange(0, 10, 2)      # [0,2,4,6,8]

print(a.shape, a.dtype, a.device)
```

Try it yourself:

1. Create a 4x4 tensor of random normal values.
 2. Create a tensor of zeros the same shape as `c` above, using `torch.zeros_like`.
 3. Convert a NumPy array `np.array([1,2,3])` into a tensor and back.
-

Module 2: Tensor Operations & Reshaping

Concept: Most bugs in deep learning code come from shape mismatches. Get comfortable reshaping, indexing, and combining tensors before moving on.

```
python
```

```

x = torch.arange(12)
x2 = x.reshape(3, 4)
x3 = x2.unsqueeze(0)      # (1, 3, 4) – add a batch dimension
x4 = x3.squeeze(0)        # back to (3, 4)

y = torch.randn(3, 4)
z = torch.randn(3, 4)
combined = torch.cat([y, z], dim=0)  # (6, 4)
stacked = torch.stack([y, z], dim=0)  # (2, 3, 4)

matmul_result = y @ z.T  # (3, 3)

```

Try it yourself:

1. Reshape a tensor of shape `(24,)` into `(2, 3, 4)`.
 2. Given two tensors of shape `(5, 3)`, stack them into shape `(2, 5, 3)`.
 3. Explain in your own words the difference between `view()` and `reshape()`.
-

Module 3: Autograd — How PyTorch Learns

Concept: `requires_grad=True` tells PyTorch to track operations on a tensor. Calling `.backward()` computes gradients automatically via the chain rule. This is the engine behind all training.

```

python

x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 3*x + 1
y.backward()
print(x.grad)  # dy/dx = 2x+3 = 7

# Gradients accumulate – must zero them each step
x.grad.zero_()

```

Try it yourself:

1. Compute the gradient of $y = 3x^3 - 2x$ at $x = 1.5$ by hand, then verify with `.backward()`.
 2. Explain why we need `optimizer.zero_grad()` at every training step.
 3. What does `torch.no_grad()` do, and when would you use it?
-

Module 4: Building a Neural Network

Concept: `nn.Module` is the base class for all models. You define layers in `__init__` and the forward pass in `forward()`. PyTorch handles gradient tracking through the whole model automatically.

python

```
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, out_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)

model = MLP(10, 32, 2)
out = model(torch.randn(4, 10))  # batch of 4
print(out.shape)  # (4, 2)
```

Try it yourself:

1. Add a `nn.Dropout(0.3)` layer between `fc1` and `fc2`.
2. Build a 3-layer MLP for input size 20, hidden sizes 64 and 32, output size 5.
3. Print `model.parameters()` and count the total number of trainable weights.

Module 5: Loss Functions & Optimizers

Concept: The loss function measures how wrong the model is. The optimizer uses gradients to update weights to reduce that loss.

python

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()          # classification
optimizer = optim.Adam(model.parameters(), lr=1e-3)

logits = model(torch.randn(4, 10))
targets = torch.tensor([0, 1, 1, 0])
loss = criterion(logits, targets)
loss.backward()
optimizer.step()
```

Try it yourself:

1. Swap `CrossEntropyLoss` for `MSELoss` — what change to the targets/output shape does this require?
2. Try `SGD` with `momentum=0.9` instead of `Adam`. What's the conceptual difference?
3. Add a learning rate scheduler (`StepLR`) and print the LR after each "epoch."

Module 6: Datasets & DataLoaders

Concept: `Dataset` defines how to get one sample; `DataLoader` batches, shuffles, and parallelizes loading. This separation keeps data logic clean.

```
python

from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

ds = MyDataset(torch.randn(100, 10), torch.randint(0, 2, (100,)))
loader = DataLoader(ds, batch_size=16, shuffle=True)

for xb, yb in loader:
    print(xb.shape, yb.shape)
    break
```

Try it yourself:

1. Add data augmentation logic inside `__getitem__` (e.g., add small random noise to `x`).
 2. Split `ds` into train/val subsets using `torch.utils.data.random_split`.
 3. Explain why `shuffle=True` matters for training but not validation.
-

Module 7: The Training Loop

Concept: Every PyTorch training loop follows the same five steps: zero gradients → forward pass → compute loss → backward pass → optimizer step.

python

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

for epoch in range(5):
    model.train()
    total_loss = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch}: loss={total_loss/len(loader):.4f}")
```

Try it yourself:

1. Add a validation pass after each epoch using `model.eval()` and `torch.no_grad()`.
 2. Track and print accuracy alongside loss.
 3. Add early stopping: stop training if validation loss doesn't improve for 3 epochs.
-

Module 8: CNNs — Image Data

Concept: Convolutional layers slide small filters over images to detect local patterns (edges, textures). Pooling reduces spatial size. Stacking conv+pool blocks builds up hierarchical features.

python

```

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(32 * 8 * 8, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 32x32 -> 16x16
        x = self.pool(F.relu(self.conv2(x))) # 16x16 -> 8x8
        x = x.flatten(1)
        return self.fc(x)

model = SimpleCNN()
out = model(torch.randn(4, 3, 32, 32))
print(out.shape) # (4, 10)

```

Try it yourself:

1. Add a `BatchNorm2d` layer after each conv layer.
2. Calculate by hand what the output spatial size would be for a `Conv2d(kernel_size=5, padding=2, stride=1)` on a 32x32 image.
3. Add a third conv block and adjust the `fc` layer's input size accordingly.

Module 9: RNNs / LSTMs — Sequence Data

Concept: RNNs process sequences step-by-step, maintaining a hidden state. LSTMs add gates to better retain long-range information and avoid vanishing gradients.

python

```

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        emb = self.embedding(x)
        _, (h_n, _) = self.lstm(emb)
        return self.fc(h_n[-1])

model = LSTMClassifier(1000, 64, 128, 2)
out = model(torch.randint(0, 1000, (4, 20)))
print(out.shape)    # (4, 2)

```

Try it yourself:

1. Make the LSTM bidirectional (`bidirectional=True`) and fix the `fc` input size.
2. Explain in your own words what the "vanishing gradient" problem is and how LSTMs address it.
3. Swap `nn.LSTM` for `nn.GRU` — what changes?

Module 10: Transfer Learning

Concept: Instead of training from scratch, reuse a model pretrained on a large dataset (like ImageNet) and fine-tune it on your smaller dataset. This saves time and usually gives better results with less data.

```

python

import torchvision.models as models

resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for param in resnet.parameters():
    param.requires_grad = False    # freeze pretrained layers

resnet.fc = nn.Linear(resnet.fc.in_features, 10)    # new head, trainable

```

Try it yourself:

1. Unfreeze only the last conv block (`resnet.layer4`) for fine-tuning.
2. Explain the difference between "feature extraction" and "fine-tuning" in transfer learning.

3. Build the optimizer so it only updates parameters with `requires_grad=True`.
-

Module 11: Transformers & Attention

Concept: Attention lets a model weigh the relevance of every other token when processing one token — instead of processing sequentially like an RNN, it looks at everything at once.

```
python

import math

def attention(Q, K, V):
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))
    weights = F.softmax(scores, dim=-1)
    return weights @ V

mha = nn.MultiheadAttention(embed_dim=64, num_heads=4, batch_first=True)
x = torch.randn(2, 10, 64)
out, attn_weights = mha(x, x, x) # self-attention
print(out.shape, attn_weights.shape)
```

Try it yourself:

1. Implement scaled dot-product attention manually and compare its output to `nn.MultiheadAttention`.
 2. Explain why we divide by `sqrt(d_k)` in the attention formula.
 3. Build a single Transformer encoder layer using `nn.TransformerEncoderLayer`.
-

Module 12: Saving & Loading Models

Concept: Save `state_dict()` (the weights), not the whole model object — it's more portable and version-safe.

```
python

torch.save(model.state_dict(), "model.pth")

model2 = MLP(10, 32, 2)
model2.load_state_dict(torch.load("model.pth"))
model2.eval()
```

Try it yourself:

1. Save a full checkpoint including optimizer state and current epoch.
2. Write code to resume training from that checkpoint.

3. Explain why `model.eval()` matters after loading for inference.
-

Module 13: GPU & Mixed Precision

Concept: Moving tensors to GPU speeds up computation. Mixed precision (using float16 where safe) speeds training further and uses less memory.

```
python

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()

for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad()
    with autocast():
        loss = criterion(model(xb), yb)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Try it yourself:

1. Time a training epoch with and without `autocast` (if you have GPU access).
 2. Explain why we need `GradScaler` alongside `autocast`.
 3. Add gradient clipping (`clip_grad_norm`) to this loop.
-

Module 14: Custom Autograd & Hooks

Concept: You can define custom forward/backward behavior, or hook into existing layers to inspect/modify gradients and activations — useful for debugging and research.

```
python
```

```

class MyReLU(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return x.clamp(min=0)
    @staticmethod
    def backward(ctx, grad_output):
        x, = ctx.saved_tensors
        grad_input = grad_output.clone()
        grad_input[x < 0] = 0
        return grad_input

activations = {}
def hook(module, input, output):
    activations["fc1"] = output.detach()
model.fc1.register_forward_hook(hook)

```

Try it yourself:

1. Implement a custom autograd `Function` for the sigmoid function.
2. Register a hook that prints the gradient flowing into `fc1` during backward.
3. Use hooks to extract intermediate features from a pretrained CNN.

Module 15: Regularization & Training Tricks

Concept: Techniques to prevent overfitting and stabilize training: dropout, weight decay, batch norm, early stopping, label smoothing.

```

python

optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

best_loss, patience, counter = float("inf"), 5, 0
for epoch in range(50):
    val_loss = 0.3 # placeholder
    if val_loss < best_loss:
        best_loss, counter = val_loss, 0
    else:
        counter += 1
        if counter >= patience:
            break

```

Try it yourself:

1. Compare training curves with and without dropout on a small dataset.
 2. Explain what "weight decay" does mathematically to the loss function.
 3. Implement early stopping that also saves the best model checkpoint.
-

Module 16: Distributed Training (Overview)

Concept: `DistributedDataParallel` (DDP) splits data across multiple GPUs/machines, each running a model copy, and synchronizes gradients after each step — the standard way to scale training.

```
python

import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group("nccl")
model = DDP(model.to(rank), device_ids=[rank])
```

Launch with: `torchrun --nproc_per_node=NUM_GPUS train.py`

Try it yourself:

1. Read about the difference between `DataParallel` (old, single-machine) and `DistributedDataParallel` (recommended).
 2. Explain what a `DistributedSampler` does and why it's needed.
-

Module 17: Exporting Models (TorchScript / ONNX)

Concept: For deployment outside Python (mobile, C++, other frameworks), export the model to a portable format.

```
python

model.eval()
scripted = torch.jit.script(model)
scripted.save("model_scripted.pt")

torch.onnx.export(model, torch.randn(1, 10), "model.onnx",
                  input_names=["input"], output_names=["output"])
```

Try it yourself:

1. Load the scripted model back with `torch.jit.load` and run inference.
 2. Explain when you'd choose TorchScript vs. ONNX.
-

Module 18: Debugging Checklist

Symptom	Likely Cause
Loss is <code>NaN</code>	LR too high, unnormalized inputs, <code>log(0)</code>
Loss doesn't decrease	Wrong loss for the task, frozen layers, LR too small
<code>CUDA out of memory</code>	Batch too large, missing <code>no_grad()</code> in eval
Shape mismatch	Wrong <code>view</code> / <code>reshape</code> , forgot <code>batch_first=True</code>
Model predicts one class always	Data not shuffled, label leakage, LR issue

python

```
torch.autograd.set_detect_anomaly(True) # find where NaNs originate (slow, de
assert torch.isfinite(loss)
```

Final Project Idea (to consolidate everything)

Build an end-to-end image classifier:

1. Load a small image dataset (e.g., CIFAR-10 via `torchvision.datasets`).
2. Write a custom `Dataset`/use built-in + `DataLoader` with augmentation.
3. Use transfer learning (ResNet18) with a frozen backbone.
4. Train with `AdamW` + `CosineAnnealingLR` + mixed precision.
5. Track train/val loss and accuracy, add early stopping.
6. Save the best checkpoint, then export to TorchScript.

Completing this project touches nearly every module above — a strong sign you're ready to move to more advanced topics (custom architectures, distributed training, research papers).